

60.6. La fonction effacerPanneau() — effacer dans le panneau

`#include <effacerPanneau>` : `effacerPanneau(colonne, ligne, quoi)` efface un mot du panneau ; sans argument, tout le panneau.

1. Ce qu'il faut comprendre

Ce qu'elle fait : comme `effacer()`, mais **dans le panneau** : `effacerPanneau(1, 0, "VIES")` vide les quatre cases du mot ; `effacerPanneau()` vide tout le panneau.

La ligne à écrire : `#include <effacerPanneau>`.

Ses arguments : rien, ou la colonne, la ligne, et le texte ou un nombre de cases.

Ce qu'elle coûte : environ **90 octets**, avec la mise en place de l'écran.

Essaie : `effacerPanneau();` : tout part.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **398 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage	– la mise en route de la console
main()	52 o	← démarrage	– ta fonction
VBlank	8 o	← la console, à chaque image	– compte les images, 60 fois par seconde
AttendreImage	28 o	← image()	– attend l'image suivante
AttendreVBlank	16 o	← démarrage	– attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte	– attend de pouvoir écrire une case
EffacerCases	12 o	← effacerPanneau()	– efface des cases
EcrireTexte	10 o	← textePanneau()	– écrit un texte, lettre par lettre
CopierTuiles	76 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
Tuiles	112 o	← CopierTuiles	– 7 lettres sur 43
"SCORE"	5 o	← textePanneau()	– un texte gravé
"VIES"	4 o	← textePanneau()	– un texte gravé

Lettres gravées : CEIORSV

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

textePanneau()	× 2	22 o sur place, dans main()	+ EcrireTexte (10 o)
panneau()	× 1	15 o sur place, dans main()	
effacerPanneau()	× 1	8 o sur place, dans main()	+ EffacerCases (12 o)
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

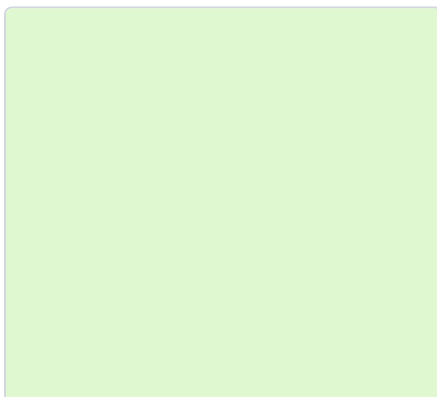
```
// ---- #include <effacerPanneau> : effacer dans le panneau ----  
// Le panneau porte SCORE et VIES ; on efface VIES, SCORE reste.  
  
#include <textePanneau>  
#include <panneau>  
#include <effacerPanneau> // effacerPanneau() : vider des cases du panneau
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

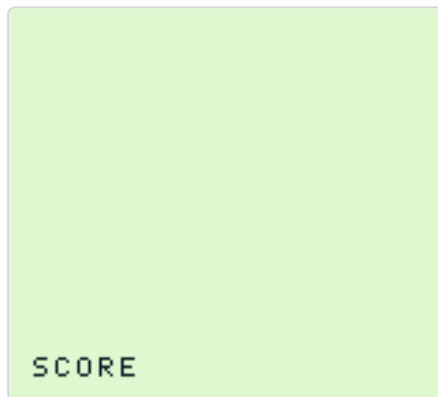
```
int main() {  
    textePanneau(1, 0, "SCORE");  
    textePanneau(10, 0, "VIES");  
    panneau(0, 128);  
    effacerPanneau(10, 0, "VIES"); // les quatre cases de VIES, comptées par le compilateur  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Le panneau ne montre plus que SCORE.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ SCORE est toujours là

✓ VIES est effacé

